

OS asssignment-10831-2022083409

Design

과제는 getppid() system call을 만들고 getpid(), getppid() system call을 사용해 학번, process id와 parent process id를 출력하는 프로그램을 만드는 것이다. 그러기 위해서 다음의 과정을 거친다.

Implementation

1. kernel directory에 prc_syscall.c 파일을 만든다.
 - 기존의 syscall.c 파일에 영향을 끼치지 않도록 별도의 파일을 만들었다. 새로운 파일이 컴파일이 될 수 있도록 Makefile에 추가한다.

```
32 | $K/prac_syscall.o
```

2. sysproc.c에 정의되어 있는 uint64 sys_getpid(void)를 참고해 uint64 sys_getppid(void)를 prac_syscall.c 파일에 정의한다.
 - proc.h 파일에 정의되어 있는 proc의 struct를 확인해 보면, parent process를 멤버로 갖고 있음을 확인할 수 있다. (struct proc *parent) myproc()은 proc.c에 정의되어 있고, 현재 process의 struct를 return한다

```
85 struct proc {
86     struct spinlock lock;
87
88     // p->lock must be held when using these:
89     enum procstate state;      // Process state
90     void *chan;                // If non-zero, sleeping on chan
91     int killed;                // If non-zero, have been killed
92     int xstate;                // Exit status to be returned to parent's wait
93     int pid;                   // Process ID
94
95     // wait_lock must be held when using this:
96     struct proc *parent;       // Parent process
97
98     // Return the current struct proc *, or zero if none.
99     struct proc*
100 myproc(void)
101 {
102     push_off();
103     struct cpu *c = mycpu();
104     struct proc *p = c->proc;
105     pop_off();
106     return p;
107 }
```

- 위를 참고해 다음과 같이 uint64 sys_getppid(void)를 만든다.

```
1 #include "types.h"
2 #include "param.h"
3 #include "memlayout.h"
4 #include "riscv.h"
5 #include "spinlock.h"
6 #include "proc.h"
7 #include "syscall.h"
8 #include "defs.h"
9
10 uint64 sys_getppid(void)
11 {
12     return myproc()->parent->pid;
13 }
14
```

3. kernel/syscall.h와 kernel/syscall.c에 다음 코드를 추가해 syscall number를 부여하고 이를 function과 연결한다.

➤ syscall.h `23 | #define SYS_getppid 22`

➤ syscall.c

`104 | extern uint64 sys_getppid(void);`

`130 | [SYS_getppid] sys_getppid,`

4. user/user.h에서 user program에서 getppid system call이 보이게 만든다.

`25 | int getppid(void);`

5. user/usys.pl를 수정해 getppid() system call을 호출하는 어셈블리 코드가 자동으로 만들어지도록 한다.

`39 | entry("getppid");`

6. user/ppid.c에서 getppid(), getpid() system call을 사용해 과제의 요구사항대로 구현한다.

```
1 #include "kernel/types.h"
2 #include "kernel/stat.h"
3 #include "user/user.h"
4 int main(int argc, char *argv[])
5 {
6     printf("My student ID is 2022083409\n");
7     printf("My pid is %d\n", getpid());
8     printf("My ppid is %d\n", getppid());
9     exit(0);
10 };
```

Result

```
xv6 kernel is booting
init: starting sh
$ ppid
My student ID is 2022083409
My pid is 3
My ppid is 2
$
```

Troubleshooting

처음엔 과제를 디테일 하게 읽지 않고 단순히 result만 같게 나오도록 코드를 짰었다. 그로 인해 다음과 같은 function들을 prac_syscall.c에 만들었었다. 이렇게 짜면 결과는 같게 나오지만, ppid만을 return하는 system call을 만든 것이 아니게 된다. 그래서 kernel에서 ppid만을 return하는 system call을 만들고 기존의 getpid() system call을 사용해 user program에서 system call을 호출하는 방식으로 방향을 바꿨다.

```
9  int myfunction(void)
10 {
11     struct proc *p = myproc();
12     printf("My studnet ID is 2022083409\n");
13     printf("My pid is %d\n", p->pid);
14     printf("My ppid is %d\n", p->parent->pid);
15     return 0;
16 }
17
18 //Wrapper for my_syscall
19 int sys_myfunction(void)
20 {
21     return myfunction();
22 }
```